

操作系统演进样本

从一台没有操作系统的机器开始

每次只解决一个已经出现的问题

这册样稿怎样展开

本册不从“操作系统由哪些模块组成”开始。起点是一台可取指、执行和访问内存，但尚未拥有操作系统的机器。每一章继承上一章结束时的机器，只处理当时已经出现的一项具体困难。

第一章只问一项任务如何开始执行；第二章才允许第二项任务进入机器；第三章再处理任务不愿主动交还处理器的情形。尚未由问题要求的结构不会提前出现。

目录

这册样稿怎样展开	ii
第一章 一台机器怎样只运行一项任务	1
1.1 通电并不等于任务已经运行	1
1.2 先让任务字节出现在内存中	1
1.3 仅有代码地址为什么还不能开始	2
1.4 任务怎样结束	2
1.5 这一方案确实能完成什么	3
第二章 第二项任务为什么需要常驻协调代码	4
2.1 直接从 A 跳到 B 会丢掉什么	4
2.2 现场保存在哪里	5
2.3 谁有资格保存现场	5
2.4 第一次运行和恢复运行为何不同	5
2.5 最小选择规则怎样形成	6
2.6 一次切换的状态账本	6
2.7 这段常驻代码已经是什么	7
第三章 任务不交还处理器时怎么办	8
3.1 常驻并不等于有机会执行	8
3.2 为什么选择周期性时钟事件	8
3.3 硬件保存的状态为什么通常不够	9
3.4 事件入口使用哪一份栈	9
3.5 怎样把事件帧并入 B 的任务记录	10
3.6 第一次出现可运行集合	10
3.7 “被唤醒”和“正在运行”尚未出现	10
3.8 一次强制切换的逐时刻状态	11
3.9 本章新增结构的准确边界	11
样稿停在这里	12

第一章 一台机器怎样只运行一项任务

本章开始时的机器。一个处理器核，一片可读写内存，一个固定复位入口，以及能把外部字节送入内存的最小固件。没有进程、线程、系统调用、文件和调度器。

1.1 通电并不等于任务已经运行

处理器能够执行指令，但它不会自行知道“今天要运行哪项任务”。通电复位只会把少量体系结构状态设为规定值，并把下一条取指位置设到一个预先约定的地址。这个地址通常落在固件可用的区域，而不是某个应用程序的开头。

先把机器能够提供的最小事实写清楚：

硬件状态	复位后提供的含义	尚未提供的含义
指令位置	下一次从哪个地址取指	哪项任务应该获得处理器
通用寄存器	若干可参与计算的位串	参数、局部变量或任务身份
内存地址	处理器可访问的字节位置	哪段是代码、数据或栈
设备入口	固件可从外部取得字节	字节属于哪个“文件”

表中右列不是硬件遗漏的缺陷，而是软件必须建立的约定。处理器只认识位模式和地址；“任务”是我们为了描述一组代码、初始数据和运行状态而使用的名称。

1.2 先让任务字节出现在内存中

假设要运行的任务已经被编译成一段机器指令和初始数据。它目前位于外部介质。最小固件必须选定一段物理内存，把这些字节复制进去。为了避免隐藏关键条件，给出一个具体布局：

物理范围	本次用途	必须保持的约束
0x100000--0x107fff	任务代码与只读常量	入口地址必须落在已装入代码中

0x108000--0x10bfff	任务可写数据	初值由任务映像给出，其余字节清零
0x10c000--0x10ffff	向下增长的栈	初始栈顶取范围末端，不能越过数据区
0x080000--0x08ffff	固件与装入代码	启动完成前不能被任务映像覆盖

这里还没有内存保护。表格描述的是装入者承诺遵守的布局，不是硬件已经阻止越界。若任务中的错误指令写到 0x080000，固件仍可能被覆盖。

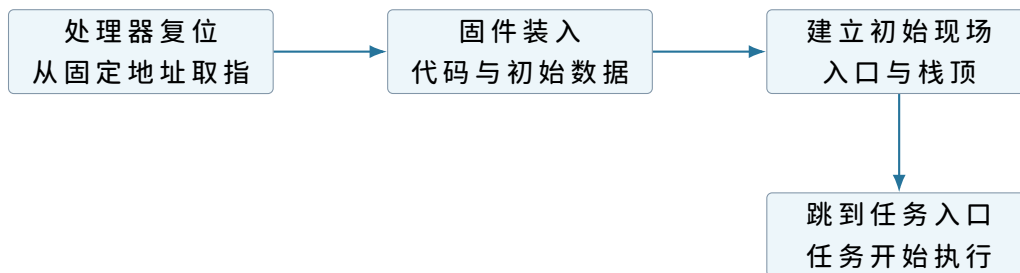
1.3 仅有代码地址为什么还不能开始

把指令写到 0x100000 后直接跳转，仍然缺少栈。普通函数调用要保存返回地址和局部状态；如果栈指针没有确定值，第一次压栈就会写入未知位置。

本次启动至少要形成以下初始现场：

```
instruction pointer = task entry address
stack pointer      = 0x110000
general registers  = documented argument values or zero
interrupt state    = the value required by the startup contract
addressing mode    = the mode used to build the task
```

“现场”在这里还不是操作系统对象。它只是处理器继续执行所需的一组具体状态。入口地址决定取哪条指令，栈指针决定调用与局部数据写到哪里，执行模式决定同一位串如何被解释。



图示：“装入”和“开始执行”是两个时刻。前者改变内存，后者改变处理器的下一条指令位置。

1.4 任务怎样结束

这台机器没有可供任务调用的退出服务。任务入口若执行普通函数返回，栈中也没有一个真实调用者留下的返回地址。最小约定可以要求任务在完成后跳到固件提供的固定返回入口：

```
task_entry:
    compute the requested result
    place result code in agreed register
    jump firmware_return
```

固件返回入口读取结果码，把处理器置于等待状态，或者重新启动同一任务。这里的“返回”完全依赖任务合作；任务必须执行那条跳转。

1.5 这一方案确实能完成什么

到此为止，机器已经能完成一条完整但很窄的路径：

1. 固件选定一个任务映像并复制到约定物理范围；
2. 固件清零未初始化数据，设置入口地址与栈顶；
3. 处理器跳到任务入口；
4. 任务直接使用全部处理器和物理内存；
5. 任务主动跳回固件入口并给出结果。

不应把这台机器描述成已经有了“单任务操作系统”。我们只建立了装入与启动约定；没有常驻任务管理对象，也没有隔离、抢占或统一服务接口。

章末出现的问题。现在有任务 A 和任务 B。运行 A 时，B 的代码放在哪里？A 完成后，谁重新装入 B 并为它建立初始现场？若想暂停 A、先运行 B、以后再继续 A，需要保存的又不只是一个入口地址。下一章只处理这项困难。

第二章 第二项任务为什么需要常驻协调代码

继承的机器。固件能装入一项任务，为它设置入口、栈和初始寄存器；任务完成时主动跳回固定入口。所有地址仍是物理地址，任务仍可访问整台机器。

2.1 直接从 A 跳到 B 会丢掉什么

设 A 正在累加一组数据。下一条指令地址保存在指令位置寄存器中，当前循环计数和部分和位于通用寄存器，函数调用关系位于 A 的栈。此时若把指令位置改成 B 的入口，处理器会立即开始取 B 的指令；但它仍带着 A 的栈顶与寄存器值。

给 B 设置自己的栈和寄存器可以让 B 正确开始，却会覆盖处理器中唯一的一份 A 状态。以后再把指令位置改回 A，并不能恢复 A，因为“不知道回到 A 的哪条指令”只是丢失状态的一部分。

必须保留的状态	丢失后的表现	为什么不能从代码映像重建
继续指令位置	A 从错误位置执行或重新开始	它取决于暂停发生的时刻
栈指针	返回地址与局部变量无法找到	栈深度随已经完成的调用变化
通用寄存器	循环变量、地址和中间结果改变	它们是运行结果，不是初始映像
状态标志	后续条件分支得到不同结果	标志来自暂停前最后一次运算
约定的扩展状态	浮点或向量计算被 B 覆盖	同样属于运行中的中间结果

这组状态可以称为任务的处理器现场。名称出现在问题之后：我们需要一份数据，能够让暂停的任务在未来像没有被中断一样继续，才有必要给这份数据命名。

2.2 现场保存在哪里

不能把 A 的现场继续留在处理器寄存器里，因为 B 就要使用这些寄存器。也不能把所有现场都压到同一栈上，再任意切换栈而不记位置。最小安排是为每项任务分配一块现场记录和一段独立栈：

```
task_state A:
  status
  resume_instruction
  stack_pointer
  general_registers
  arithmetic_flags
  stack_low, stack_high

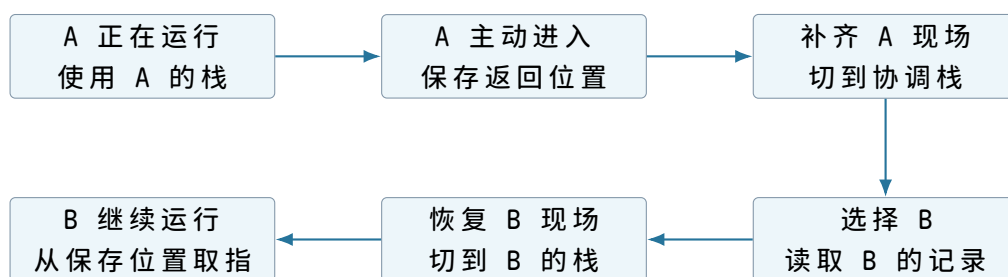
task_state B:
  same fields, different storage
```

`status` 至少区分“尚未开始”“可以继续”“正在运行”和“已经完成”。它不是为了美化结构，而是避免协调代码把一份只有初始入口、尚无暂停现场的记录当作可恢复记录。

2.3 谁有资格保存现场

若 A 自己把寄存器依次写入记录，会遇到一个细节：执行保存代码本身也需要寄存器，而且“保存后的继续地址”应当指向 A 交还处理器之后的位置。最小方案规定一个调用约定：A 像调用函数一样进入协调代码，调用约定先把一部分状态压到 A 的栈，协调代码再补存其余状态并记录当前栈指针。

协调代码必须常驻在不会被任务映像覆盖的内存中。它还需要一个只在选择任务和恢复现场时使用的协调栈，否则恢复 A 的栈之后再运行复杂选择代码，会误把协调者的局部数据写进 A。



图示：切换不是把 A 的代码换成 B 的代码，而是先把处理器唯一的活动现场写回 A 的记录，再把 B 的记录装入处理器。

2.4 第一次运行和恢复运行为何不同

B 第一次获得处理器时，没有“暂停位置”。它的记录由装入者人工构造成一种可恢复形态：继续位置设为 B 的入口，栈指针设为 B 的初始栈顶，其余寄存器按启动约定给值。以后 B 主动交还处理器，记录才被真实运行状态覆盖。

B 的阶段	记录中的继续位置	栈中已经存在的内容
尚未第一次运行	任务入口	人工准备的初始参数或空栈
运行后主动交还	交还入口之后的继续点	B 已经形成的调用链与局部数据
已经完成	不再作为可恢复位置使用	可等待回收或检查结果

统一的恢复代码只读取记录，不必在最后一步区分“第一次”和“后来”。区别已经在记录如何构造、状态字段是否允许恢复中表达。

2.5 最小选择规则怎样形成

当前只有 A 和 B。协调代码可使用一个当前编号：A 交还时若 B 可继续，就选择 B；B 交还时再选择 A。这个规则尚不需要复杂队列，但仍要处理任务完成：

```
choose_next:
    inspect the other task
    if it is ready:
        return that task
    if current task only yielded and remains ready:
        return current task
    otherwise:
        wait for an external event
```

这里第一次出现“选择下一项可运行任务”的职责。由于选择只发生在任务主动进入协调代码后，它仍是合作式安排：协调者能决定交还之后运行谁，却不能决定任务何时交还。

2.6 一次切换的状态账本

时刻	处理器活动现场	A 的内存记录	B 的内存记录
t_0	A 的状态	旧值，不可用于恢复 当前 A	B 的初始或暂停状态
t_1	A 正在执行保存入口	部分字段已更新	不变
t_2	协调代码使用协调栈	A 的完整暂停状态	不变
t_3	正在装入 B 的状态	完整且稳定	被读取，尚未运行
t_4	B 的活动现场	A 可恢复	B 记录成为旧值

任一时刻，正在运行任务的最新寄存器在处理器中，其内存记录只是上一次保存值；暂停任务的最新现场则必须完整位于内存记录和它自己的栈中。说“每项任务都有一份现场”时，需要同时说明活动现场究竟位于处理器还是内存。

2.7 这段常驻代码已经是什么

它不是完整操作系统，却已经具有三个此前不存在的职责：保存当前任务、选择下一任务、恢复所选任务。由于任务运行期间协调代码仍保留在内存中，可以把它称为常驻协调者。以后若加入更多任务，选择规则和记录集合会扩展，但本章不提前设计最终结构。

章末出现的问题。 A 可以约定每处理一批数据就主动交还处理器；B 却可能进入无限循环，或者因为错误永远不调用交还入口。此时协调代码虽然常驻，却没有任何机会执行。下一章只处理“机器怎样在任务不合作时重新得到控制”。

第三章 任务不交还处理器时怎么办

继承的机器。常驻协调者能保存、选择和恢复 A/B；每项任务有独立现场记录与栈。切换入口只能由正在运行的任务主动调用。

3.1 常驻并不等于有机会执行

B 执行下面的指令序列：

```
repeat_forever:
  compute
  jump repeat_forever
```

处理器会不断从 B 的地址取指。协调代码虽然存在于另一段内存，但“存在”不会自动改变指令位置。外部观察者也不能靠在内存里写一个标志解决问题，因为 B 没有读取该标志。

要在 B 不执行交还指令时重新获得控制，机器必须提供一种与当前指令流独立的事件：事件到达后，处理器先保存足以继续 B 的状态，再把指令位置改成预先登记的入口。

3.2 为什么选择周期性时钟事件

设备完成也能产生外部事件，但机器空闲时未必有设备活动，不能用它保证处理器一定被收回。一个可编程计时器可以在设定时间后发出事件，与 B 是否读内存、是否调用函数无关。

为支持这条路径，硬件至少要明确：

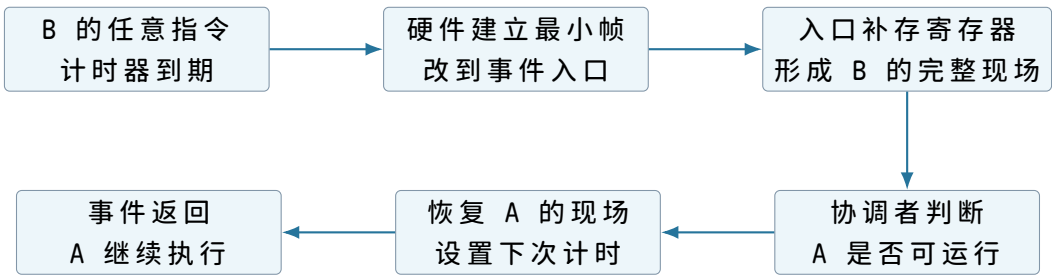
硬件能力	必须提供的事实	软件随后才能做的事
计时器	到期时产生一个可识别事件	规定最长连续运行区间
事件入口表	事件编号对应固定入口地址	让协调代码得到控制

最小现场保存	保存被打断继续位置与必要标志	以后从原指令流继续
事件屏蔽状态	防止入口最早阶段被同类事件反复覆盖	完成一次一致的现场建立
事件返回指令	从规定帧恢复状态与执行级别	回到所选任务

硬件不必替软件决定运行 A 还是 B。它只保证事件发生时能离开 B，并把控制交到一个软件已知入口。选择仍由协调代码完成。

3.3 硬件保存的状态为什么通常不够

事件到达的第一目标是安全进入入口，而不是一次完成所有任务切换。处理器通常只自动保存继续位置、状态标志以及进入和返回所必需的少量字段。通用寄存器若要保持不变，入口软件仍要尽早保存。



图示：时钟事件只强制打开入口机会。完整保存、任务选择和恢复仍是软件协议。

入口不能先使用尚未保存的通用寄存器计算，因为那会覆盖 B 的值。最早几条指令只能使用硬件约定允许破坏的状态，或者把寄存器按固定顺序压入已知安全的栈。

3.4 事件入口使用哪一份栈

B 的栈可能已接近边界，甚至因为错误把栈指针改成无效值。若硬件入口无条件继续使用 B 的栈，保存第一项状态就可能失败。机器需要一个不由普通任务控制的入口栈位置；单核样机可让硬件或最早入口从固定协调状态取得它。

栈	保存的调用关系	所有权边界
A 的任务栈	A 的函数返回地址与局部数据	A 运行或暂停时保留
B 的任务栈	B 的函数返回地址与局部数据	B 运行或暂停时保留
协调入口栈	事件最早保存与选择代码 局部状态	普通任务不应改写

入口栈解决的是“如何安全开始保存”，不是自动隔离。由于当前机器仍允许 B 写任意物理地址，B 依然能够故意或错误地覆盖协调入口栈；这个缺陷留到章末。

3.5 怎样把事件帧并入 B 的任务记录

事件帧记录 B 被打断的位置，软件补存通用寄存器。协调者可以让 B 的任务记录指向这份完整帧，而不必把每个字段再复制一次。关键是记录必须说明帧位于哪一段栈、布局采用哪个入口约定。

```
timer_entry:
  hardware has saved resume position and flags
  save remaining registers in fixed order
  current_task.saved_stack = current stack pointer
  current_task.status = ready
  switch to coordinator stack
  next = choose_ready_task()
  restore next.saved_stack
  return_from_event
```

主动交还与时钟打断可以最终汇入相同的“已形成完整现场”边界，但两者最早入口不同。主动交还遵守普通调用约定；时钟可以发生在任意指令，必须按异步入口约定保存所有可能被后续代码破坏的状态。

3.6 第一次出现可运行集合

只有两项任务时也需要区分“暂停但能继续”和“已经完成”。协调者不应把完成的任务重新装入处理器。可以维护一个很小的可运行集合：

任务状态	是否参与选择	状态改变的入口
ready	是	被时钟打断或主动交还后
running	否，当前已占用处理器	协调者恢复该任务时
finished	否	任务进入完成入口时
not started	可先构造初始现场再加入	装入者完成代码、数据和栈准备时

选择规则可以是 A、B 轮流。每次恢复前重新设置计时器，给所选任务一个有限运行区间。时间区间结束并不表示任务完成，只表示协调者获得一次重新选择的机会。

3.7 “被唤醒”和“正在运行”尚未出现

本章任务只做计算，没有等待设备或条件；ready 已足够表达“具有运行资格”。以后加入等待后，某项任务即使没有完成也可能暂时不应进入选择集合，届时才需要额外状

态和唤醒协议。现在提前画完整生命周期只会让尚无问题来源的概念进入样机。

3.8 一次强制切换的逐时刻状态

时刻	控制流位置	B 的最新现场	A 的最新现场
t_0	B 的循环	处理器中	A 的内存记录与栈
t_1	硬件事件入口	最小字段在事件帧， 其余仍在寄存器	不变
t_2	软件入口完成保存	B 的完整帧与栈	不变
t_3	协调者选择 A	稳定，可供以后恢复	正在被读取
t_4	事件返回到 A	B 的内存现场	处理器中

事件返回的目标不必是刚才被打断的 B。硬件要求返回帧格式正确；软件可以在返回前换成 A 的已保存帧。这个能力使时钟入口成为抢占切换点。

3.9 本章新增结构的准确边界

这一阶段只新增：

- 可编程计时器及其事件入口；
- 不依赖普通任务栈的最早入口状态；
- 能容纳异步打断的完整任务现场格式；
- ready/running/finished 三类选择状态；
- 每次恢复前重新设置时间预算的轮流选择规则。

它没有新增文件、用户态、系统调用、虚拟地址、权限或设备服务。所谓“最小抢占调度”在此只表示：协调者能在任务未主动交还时获得入口，并从可运行集合选择另一项任务。

章末出现的问题。 B 已经不能永久独占处理器，却仍能执行一条写指令覆盖 A 的栈、A 的代码、协调者记录或时钟入口。时间上的轮流使用已经建立，空间上的互不破坏仍未建立。下一次演进应从这次真实覆盖失败出发，再决定硬件与软件需要增加什么；本样稿在此停止。

样稿停在这里

此时机器已经能装入多项任务，并能依靠时钟事件收回处理器。然而任务仍共享物理地址，任何任务都能覆盖协调代码或其他任务。这是下一次演进应处理的问题；本样稿暂不提前给出答案，等待对叙事粒度与技术深度进行评审后再继续。